

CINEBOOK

APP AGENT

DATABASE ENGINE AGENT

QA AGENT



GOOD
MOVIES
BRIGHTER
PEOPLE



VERCEL DEVELOPMENT PROMPT

Next.js | Neon PostgreSQL | Drizzle ORM | Three-Agent Team

CineBook Development Specification

Create a team called CineBook with three agents.

Goal: Build a production-ready cinema ticket-booking web app and prepare it for deployment on Vercel.

Technology

- Next.js with TypeScript and App Router
- Tailwind CSS
- Neon serverless PostgreSQL from Vercel Marketplace
- Drizzle ORM with migrations
- Secure authentication
- Vercel Functions for backend operations
- Vercel Blob for uploaded movie images if needed
- Payment provider in test mode
- Never expose database or payment secrets in browser code

Agent 1 - App Agent

- Build a polished responsive cinema-booking interface
- Create home, movie details, cinema, showtime, seat selection, checkout, ticket, booking history, login, and admin pages
- Let users search and filter movies by title, genre, language, cinema, and date
- Show accurate seat availability for each showtime
- Display selected seats, fees, and final total before payment
- Generate a digital ticket with booking reference and QR code
- Allow users to view and cancel eligible bookings
- Connect every screen to the real database
- Do not use permanent mock data or placeholder buttons
- Optimize the application for desktop and mobile

Agent 2 - Database Engine Agent

Use Neon PostgreSQL and create migrations for:

- users
- movies
- genres
- movie_genres
- cinemas
- auditoriums
- seats
- showtimes
- showtime_seats
- bookings
- booking_items
- payments
- tickets
- audit_logs

Database Rules

- Use UUID primary keys
- Store all timestamps in UTC
- Store money as integer minor units, never floating-point values
- Add foreign keys, validation constraints, indexes, and appropriate cascade rules
- Create unique constraints for cinema screen names and auditorium seat positions
- Create a unique constraint that prevents the same showtime seat from being booked twice
- Store seat status as AVAILABLE, HELD, BOOKED, or BLOCKED
- Store booking status as PENDING, CONFIRMED, CANCELLED, EXPIRED, or REFUNDED
- Store payment status separately from booking status

Booking Transaction

1. Begin a database transaction
2. Lock the requested showtime-seat records
3. Confirm every requested seat is available
4. Create a temporary seat hold with an expiration time
5. Calculate the price on the server
6. Create the pending booking
7. Commit the transaction
8. Confirm seats only after verified payment
9. Use payment idempotency keys so retries cannot create duplicate bookings
10. Reject the booking if any selected seat is no longer available

Create a secure server-side endpoint for releasing expired seat holds. Make the operation idempotent so repeated runs are safe. If using Vercel Cron, protect the route with **CRON_SECRET**.

Add seed data containing sample movies, cinemas, auditoriums, seats, and showtimes.

Agent 3 - QA Agent

- Test registration, login, logout, and authorization
- Test movie search, cinema selection, showtimes, and seat maps
- Test complete bookings from movie selection through ticket confirmation
- Attempt to book the same seat simultaneously from two sessions
- Confirm only one booking succeeds
- Test expired seat holds and automatic seat release
- Verify totals, taxes, service fees, and payment status
- Test payment success, failure, cancellation, retry, and duplicate webhook delivery
- Confirm users cannot access another user's bookings or tickets
- Test admin permissions
- Test database migrations on a clean database
- Test Vercel preview and production builds
- Test desktop and mobile layouts
- Report every bug to the responsible agent and retest each fix

Deployment Requirements

- Deploy the Next.js application to Vercel
- Provision Neon PostgreSQL through Vercel Marketplace
- Use pooled DATABASE_URL for application requests
- Keep migration credentials and production secrets server-side
- Configure separate preview and production environments
- Run migrations safely during deployment
- Configure payment webhook secrets
- Protect scheduled cleanup endpoints
- Add error handling, loading states, empty states, and structured server logs
- Create a README containing local setup, environment variables, migrations, testing, and Vercel deployment instructions

The agents must share a task list and communicate directly. Do not mark the project complete until the database migrations pass, concurrent seat-booking tests pass, payment flows work in test mode, and the deployed Vercel application works end to end.